REACH: A Natural Language Processing-Based Reviewer Matching and Conference Management Framework for Double-Blind University Research Conferences

1.0 Executive System Overview & Scope

1.1 System Overview

REACH (Reviewer Evaluation and Assignment for Conference Handling) is a web-based university research-conference management framework whose primary Computer Science research contribution is an **NLP-based reviewer-matching component**.

The system addresses the difficulty of manually matching submitted research papers with appropriate reviewers while simultaneously considering reviewer expertise, workload capacity, conflicts of interest (COI), and double-blind review requirements.

The project is specifically situated within the **Quezon City University (QCU) research-conference context**, including activities such as **IRISE, Synergy, Pioneer, and the Business Research Colloquium**. The QCU proposal identifies fragmented conference administration through basic online forms and cloud storage as a practical problem requiring substantial manual coordination.

The conference portal is therefore treated as the **operational testbed** for the Computer Science research rather than as the principal research contribution. The proposal explicitly establishes that the computational contribution is the comparison of **keyword-based matching against TF-IDF with cosine similarity**, with workload and COI rules applied to reviewer recommendations.

1.2 Core Problem

The current problem can be decomposed into four related operational issues:

1. **Fragmented conference administration**
    - Research submissions may be collected through separate online forms.
    - Supporting documents may be maintained in cloud storage.
    - Reviewer information may be maintained separately.
    - Status tracking and communication require manual coordination.
2. **Manual reviewer matching**

    - Organizers must inspect paper topics.
    - Reviewer expertise must be manually compared against research topics.
    - Matching becomes increasingly difficult as submission volume increases.
3. **Workload and conflict constraints**
    - A reviewer cannot indefinitely accept new assignments.
    - Reviewers may have declared conflicts of interest.
    - Selecting only the highest textual similarity does not guarantee a valid assignment.
4. **Double-blind review requirements**
    - Reviewers must not unnecessarily receive author-identifying information.
    - Authors must not be exposed to reviewer identities through the review workflow.
    - Administrative users require controlled access to identifying information.

The QCU proposal specifically establishes that reviewer assignment must account for **expertise, workload limits, and conflicts of interest**, rather than treating similarity alone as sufficient.

1.3 Proposed Solution

REACH shall provide a centralized web-based environment consisting of:

- Conference and Call-for-Papers management
- Research submission
- Reviewer profile and expertise management
- Reviewer workload and capacity management
- Conflict-of-interest declarations
- NLP-based reviewer recommendation
- Double-blind review routing
- Review and revision management
- Acceptance-status management
- Research repository
- Conference analytics
- Matching-experiment and evaluation facilities
- Audit logging

The system shall calculate textual compatibility between:

Submitted Research Paper ↔ Reviewer Expertise Profile

using two experimentally evaluated approaches:

1. **Keyword-Based Matching**
2. **TF-IDF + Cosine Similarity**

The resulting scores shall be subjected to deterministic constraints before reviewer recommendations are presented to the conference administrator.

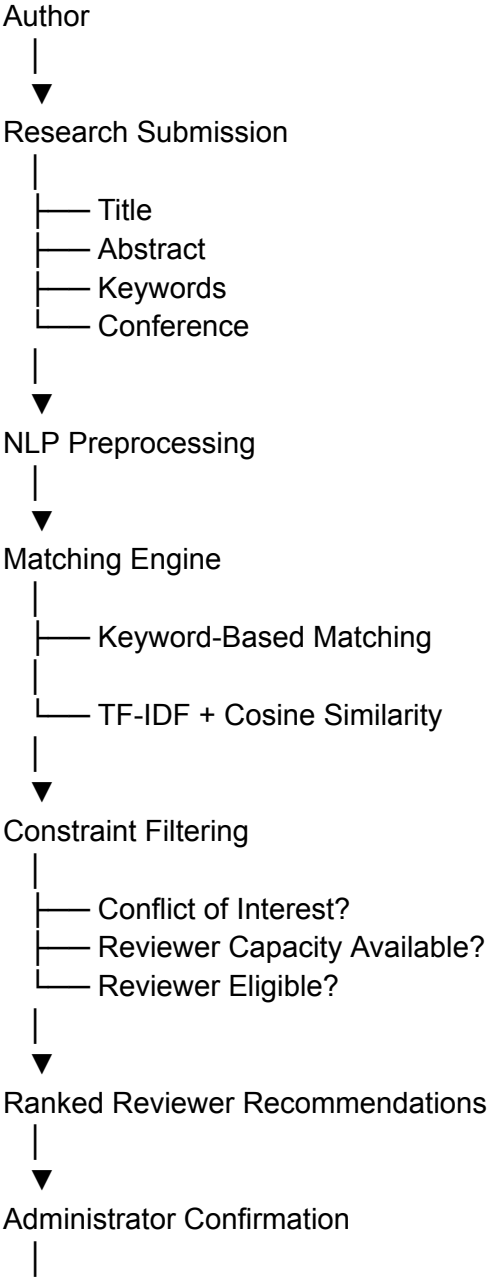
1.4 Primary Research Contribution

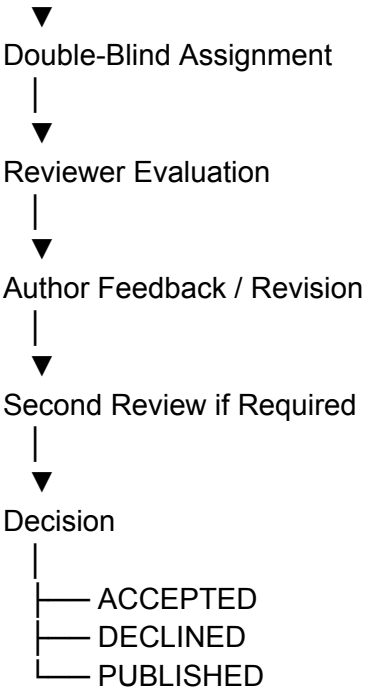
The primary research contribution is:

> **An explainable and experimentally evaluable NLP-based reviewer-matching mechanism that compares keyword-based matching with TF-IDF/cosine similarity while enforcing practical reviewer workload and conflict-of-interest constraints within a double-blind university conference workflow.**

The proposal explicitly identifies Precision@K, workload standard deviation, and processing time as the principal evaluation measures.

1.5 Operational Workflow





1.6 System Scope

The digital workflow shall cover:

Functional Area	Included
Conference creation	Yes
Call for Papers	Yes
Participant registration	Yes
Research submission	Yes
Abstract and keyword storage	Yes
Reviewer profile management	Yes
Reviewer expertise	Yes
Reviewer workload capacity	Yes
Conflict-of-interest declarations	Yes
NLP reviewer matching	Yes
Reviewer recommendations	Yes
Administrator assignment confirmation	Yes

Double-blind routing	Yes
Review submission	Yes
Revision workflow	Yes
Second review	Yes
Acceptance/decline recording	Yes
Research repository	Yes
Analytics	Yes
Audit logs	Yes

The workflow corresponds to the approved proposal's stated scope covering call for papers, registration, submission, reviewer assignment, peer review, revision, acceptance, and repository management.

1.7 Absolute Delimitations

The following features are **strictly outside the project scope**:

- Deep-learning model training
- Large Language Model training
- Custom machine-learning model development
- AI-generated-text detection
- Automated plagiarism detection
- AI-generated research grading
- Automated research-quality judgment
- Automatic paper acceptance
- Automatic paper rejection
- Fully autonomous reviewer assignment without administrator confirmation
- Complex mathematical optimization solvers
- IoT hardware deployment
- Blockchain
- Cryptocurrency/payment processing
- Ticketing
- Hotel/accommodation registration
- Live video conferencing
- Commercial conference-management features

The approved proposal explicitly excludes automated acceptance/rejection, AI grading, deep learning, LLM training, custom ML development, plagiarism/AI-text detection, commercial conference features, and custom IoT infrastructure.

1.8 System Authority Model

REACH shall follow the principle:

> **NLP recommends; deterministic rules filter; the administrator decides.**

The system shall **never make the final acceptance decision or final reviewer assignment autonomously**.

The administrator remains responsible for:

- Reviewing recommendations
- Confirming assignments
- Monitoring workload
- Resolving exceptional cases
- Managing conference decisions
- Authorizing publication

2.0 IEEE-Aligned SDLC Implementation Roadmap

2.1 SDLC Governance

The project shall follow a structured software-engineering lifecycle consisting of:

1. Requirements Analysis
2. System and Software Design
3. Implementation
4. Integration
5. Verification and Testing
6. Evaluation
7. Deployment
8. Documentation and Maintenance

The lifecycle shall use an iterative development approach internally while maintaining formal engineering documentation required for the thesis.

Each phase shall produce measurable artifacts and acceptance criteria.

2.2 Academic Project Timeline

The QCU proposal establishes an **August 2026–May 2027** project period. Its Gantt chart begins with requirements/problem identification, literature review, proposal approval, and Chapters 1–3, followed by Chapters 4–5, oral defense, revision, and hardbound/software submission.

Master Implementation Timeline

Phase	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
Requirements & Problem Analysis	●	●								
Literature Review	●	●	●							
Proposal Approval	●	●								
Chapter 1	●	●								
Chapter 2		●	●							
Chapter 3			●	●						
Data-Gathering Design			●	●						
System Architecture			●	●						
Database Design			●	●						
UI/UX Design			●	●						
Core Implementation				●	●	●				
NLP Implementation				●	●	●				
Portal Integration				●	●	●	●			
Testing						●	●	●		
Empirical Evaluation							●	●		
Chapter 4							●	●		
Chapter 5								●	●	
Defense Preparation								●	●	
Oral Defense									●	
Revisions									●	
Hardbound + Software Submission									●	

2.3 Phase 1 — Requirements Analysis

Target: August–September 2026

Activities

- Identify current conference-management problems.
- Analyze existing QCU conference workflows.
- Define system stakeholders.
- Identify user roles.
- Document functional requirements.
- Document non-functional requirements.
- Establish scope and delimitations.
- Define NLP experiment requirements.
- Define evaluation dataset requirements.
- Define security and privacy requirements.
- Define double-blind requirements.

Required Deliverables

- Problem statement
- Stakeholder analysis
- Scope and delimitation
- Functional requirements specification
- Non-functional requirements specification
- Initial use-case model
- Data requirements
- Evaluation requirements
- Chapter 1 draft

Phase Exit Criteria

Requirements are considered approved when:

- Every system function has an identified stakeholder.
- Every requirement has a verification method.
- Out-of-scope functionality is documented.
- NLP evaluation metrics are finalized.
- Double-blind privacy requirements are defined.

3.0 Requirements Specification

3.1 User Roles

Administrator

The administrator shall:

- Create conferences
- Publish announcements
- Configure CFP information
- Review submissions
- Manage reviewers
- Configure reviewer capacity
- Review COI declarations
- Generate reviewer recommendations
- Confirm assignments
- Monitor reviews
- Manage revisions
- Record decisions
- Publish accepted papers
- Access analytics
- Access audit logs

Author / Researcher

The author shall:

- Register
- Maintain profile information
- Submit research
- Provide title, abstract, and keywords
- Track submission status
- Receive reviewer feedback
- Submit revisions
- View final decision
- Access published work where applicable

Reviewer

The reviewer shall:

- Maintain expertise information
- Define research interests
- Provide keywords
- Declare conflicts of interest
- Define/accept workload capacity
- View assigned anonymized submissions
- Submit reviews
- Provide comments
- Evaluate revisions when assigned

Public Viewer

The public viewer shall:

- View published conferences
- View public announcements
- View published research
- Access non-confidential conference information

The public viewer shall not access:

- Author identities before publication
- Reviewer identities
- Private reviews
- Internal assignments
- COI records
- Administrative audit records

4.0 System Functional Requirements

4.1 Authentication and Authorization

The system shall provide:

- Secure login
- Role-based access control
- Session management
- Password/account recovery through the selected authentication provider
- Role-specific route protection
- Server-side authorization checks

4.2 Conference Management

The administrator shall be able to:

- Create conferences
- Define conference title
- Define dates
- Define submission deadlines
- Publish CFP
- Create announcements
- Open/close submissions
- Configure review periods

4.3 Submission Management

Each submission shall support:

- Submission identifier
- Conference
- Title
- Abstract
- Keywords
- Author records
- Uploaded manuscript where required
- Submission date
- Status
- Revision records

Example blinded identifier:

Submission #2027-001

The reviewer interface shall display the submission identifier rather than author identity.

4.4 Reviewer Management

Reviewer profiles shall contain:

- Reviewer account
- Expertise description
- Expertise categories
- Keywords
- Research interests
- Capacity
- Current workload
- COI declarations
- Availability status

4.5 Review Workflow

Supported statuses shall include:

SUBMITTED
↓
UNDER REVIEW
↓
REVISION REQUIRED
↓
RESUBMITTED
↓
UNDER SECOND REVIEW
↓
ACCEPTED / DECLINED
↓

PUBLISHED

These statuses are consistent with the established project workflow and its objective of supporting review, revision, acceptance, and repository management.

5.0 NLP Algorithm & Core Logic Specifications

5.1 NLP Research Objective

The NLP component shall determine how effectively textual similarity can support reviewer recommendation.

The experiment shall compare:

Method A — Keyword-Based Matching

A simple lexical baseline based on overlap between:

- Submission keywords
- Reviewer keywords/expertise terms

Method B — TF-IDF + Cosine Similarity

A vector-space representation of submission and reviewer expertise text followed by cosine similarity calculation.

The QCU proposal explicitly establishes this comparison as the core computational investigation.

5.2 Input Data

Submission Text

The NLP system shall extract:

- Title
- Abstract
- Author-provided keywords

Reviewer Text

The NLP system shall extract:

- Expertise description
- Research interests
- Reviewer keywords
- Expertise categories where applicable

The proposal specifically identifies paper titles, abstracts, keywords, and reviewer expertise information as the NLP inputs.

5.3 Text Preprocessing

The preprocessing pipeline shall be deterministic.

Raw Text
↓
Lowercase
↓
Remove Punctuation
↓
Tokenization
↓
Stopword Removal
↓
Optional Stemming/Lemmatization
↓
Normalized Token Set/Text

The preprocessing configuration shall be identical for both experimental methods wherever applicable.

Example

Input:

> Machine Learning-Based Flood Risk Prediction Using GIS

Normalized representation:

machine
learning
based
flood
risk
prediction
gis

The exact preprocessing configuration shall be documented in the methodology so the experiment is reproducible.

6.0 Keyword-Based Matching

6.1 Purpose

Keyword matching serves as the **baseline method**.

It establishes whether a simple lexical matching mechanism can provide useful reviewer recommendations before introducing TF-IDF.

6.2 Basic Procedure

For every submission:

1. Extract normalized submission keywords.
2. Extract normalized reviewer keywords.
3. Compare the two sets.
4. Count overlapping terms.
5. Calculate the keyword matching score.
6. Rank eligible reviewers.

A basic score can be represented as:

KeywordScore=∣Ks∩Kr∣∣Ks∣KeywordScore = \frac{|K_s \cap K_r|} {|K_s|}

Where:

- KsK_s = submission keyword set
- KrK_r = reviewer keyword set
- Ks∩KrK_s \cap K_r = shared keywords

The final implementation shall document the exact scoring definition used by the experiment.

6.3 Explainability

The system shall display:

- Matching score
- Shared keywords
- Reviewer expertise
- Current workload
- Eligibility status

This allows the administrator to understand why a reviewer was recommended.

7.0 TF-IDF + Cosine Similarity

7.1 Purpose

TF-IDF shall provide a vector representation capable of distinguishing terms based on their frequency within documents and rarity across the evaluation corpus.

7.2 TF-IDF

Term Frequency:

TF(t,d)=Number of occurrences of t in dTotal terms in dTF(t,d)= \frac{\text{Number of occurrences of }t\text{ in }d} {\text{Total terms in }d}

Inverse Document Frequency:

IDF(t)=log(Ndf(t))IDF(t)= \log \left(\frac{N} {df(t)} \right)

TF-IDF:

TFIDF(t,d)=TF(t,d)×IDF(t)TFIDF(t,d)=TF(t,d)\times IDF(t)

Where:

- tt = term
- dd = document
- NN = number of documents
- df(t)df(t) = number of documents containing the term

7.3 Cosine Similarity

The similarity between a submission vector AA and reviewer profile vector BB shall be:

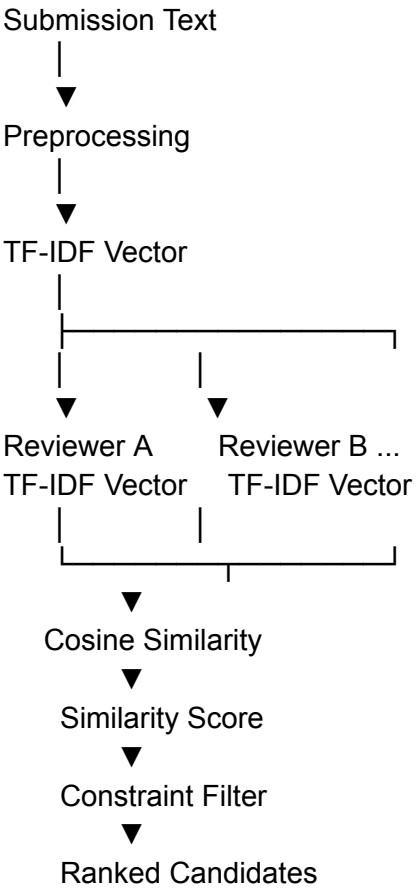
Cosine(A,B)=A⋅B // A // // B // Cosine(A,B)= \frac{A\cdot B} {\|A\|\|B\|}

The resulting score shall normally fall within:

0≤Similarity≤10 \leq Similarity \leq 1

for the non-negative TF-IDF vectors used by the system.

7.4 Matching Procedure



8.0 Deterministic Constraint Rules Engine

8.1 Design Principle

Similarity scoring shall **not override hard constraints**.

The processing order shall be:

Similarity Calculation
↓
Eligibility Filtering
↓
COI Filtering
↓
Workload Filtering
↓
Ranking
↓
Administrator Review
↓

Assignment Confirmation

8.2 Conflict-of-Interest Rule

If:

Reviewer ↔ Submission = Declared COI

then:

Reviewer Eligibility = FALSE

The reviewer shall be removed from the candidate pool regardless of similarity score.

8.3 Workload Rule

If:

Current Reviewer Workload >= Maximum Reviewer Capacity

then:

Reviewer Eligibility = FALSE

A reviewer with a similarity score of `0.99` shall still be excluded if the workload constraint is violated.

8.4 Eligibility Function

Conceptually:

Eligible(r,s)=COI(r,s)=0∧Workload(r)<Capacity(r)Eligible(r,s)= COI(r,s)=0 \land Workload(r)<Capacity(r)

Only eligible reviewers shall proceed to the final ranking stage.

8.5 Ranking

For the remaining eligible reviewers:

RankedReviewers=sort(SimilarityScore, descending)RankedReviewers = sort(SimilarityScore,\ descending)

The system shall present the highest-scoring eligible reviewers first.

8.6 Administrator Confirmation

The algorithm shall produce **recommendations**, not irrevocable assignments.

The administrator shall be able to:

- Confirm recommendation
- Select another eligible reviewer
- View explanation
- Review workload
- Inspect applicable COI status
- Re-run recommendations after profile changes

9.0 Reviewer-Matching Experimental Design

9.1 Experimental Objective

Determine which of the two matching approaches provides better reviewer-research relevance while operating under identical practical constraints.

9.2 Controlled Variables

The experiment shall maintain:

- Same submission dataset
- Same reviewer dataset
- Same preprocessing policy
- Same COI constraints
- Same workload constraints
- Same number of reviewers where applicable
- Same relevance judgments
- Same evaluation procedure

Only the matching method shall change.

9.3 Experimental Conditions

Condition	Method	Constraints

A	Keyword-Based Matching	COI + workload
B	TF-IDF + Cosine Similarity	COI + workload

9.4 Evaluation Dataset

The proposal requires an evaluation dataset consisting of publicly available research abstracts and reviewer expertise information, with relevance judgments validated by selected domain experts.

The dataset shall contain, where possible:

- Research identifier
- Title
- Abstract
- Keywords
- Research topic/category
- Reviewer identifier
- Reviewer expertise
- Reviewer keywords
- Relevance judgment

Personal identifying information shall not be required for the experimental dataset.

10.0 Evaluation Metrics

10.1 Precision@K

Precision@K shall measure the proportion of relevant reviewers among the top K recommendations.

Precision@K=Relevant reviewers in top KKPrecision@K = \frac{\text{Relevant reviewers in top K}} {K}

Example:

If 3 of the top 5 recommended reviewers are judged relevant:

Precision@5=35=0.60Precision@5=\frac{3}{5}=0.60

The same K values shall be applied consistently across methods.

10.2 Workload Standard Deviation

Workload distribution shall be evaluated using standard deviation.

σ=1N∑i=1N(xi−x¯)2\sigma = \sqrt{ \frac{1}{N} \sum_{i=1}^{N} (x_i-\bar{x})^2 }

Where:

- xix_i = reviewer workload
- x¯\bar{x} = average workload
- NN = number of reviewers

Lower standard deviation generally indicates a more evenly distributed workload.

The proposal explicitly identifies standard deviation for workload variance as one of the principal evaluation metrics.

10.3 Processing Time

The system shall record the elapsed processing time required to produce reviewer recommendations.

At minimum:

Start timestamp
↓
NLP processing
↓
Similarity calculation
↓
Constraint filtering
↓
Ranking
↓
End timestamp

ProcessingTime=EndTime−StartTimeProcessingTime = EndTime-StartTime

The same computing environment and test conditions shall be used when comparing methods wherever practical.

10.4 Primary Evaluation Table

Metric	Purpose	Desired Interpretation
Precision@K	Reviewer relevance	Higher is better
Workload SD	Assignment distribution	Lower indicates more even distribution

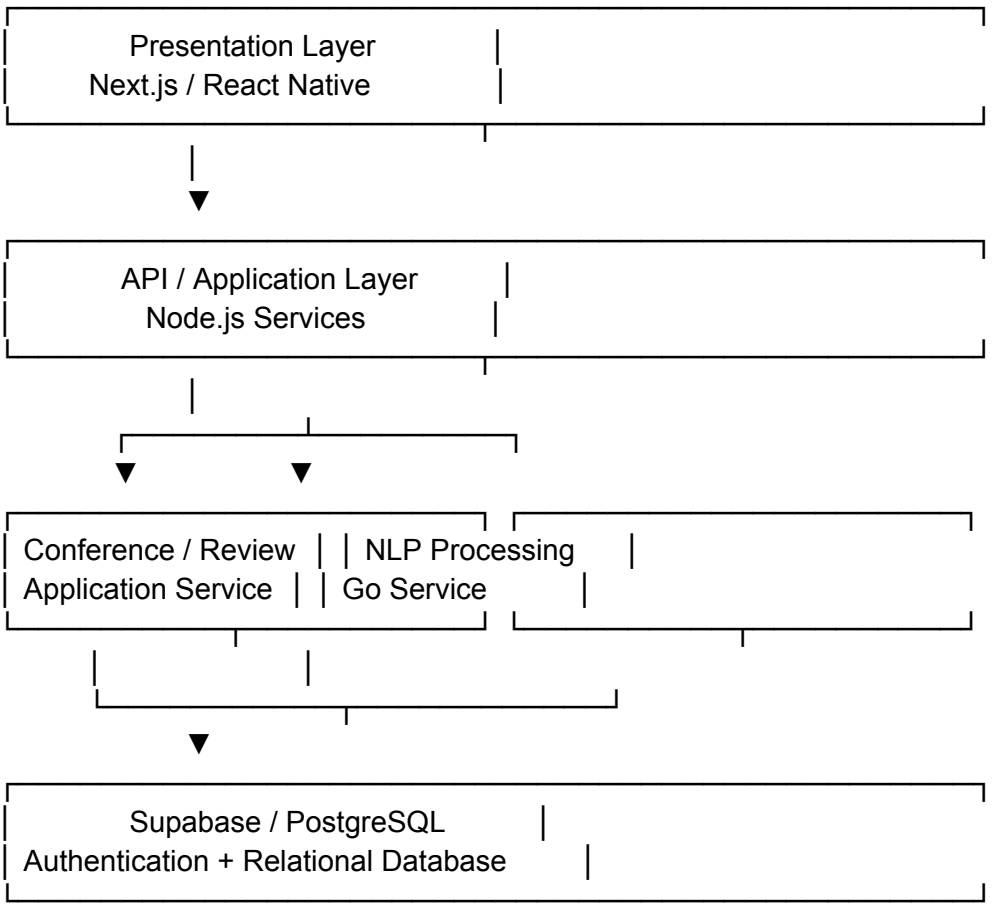
Processing Time	Computational efficiency	Lower is generally better

These metrics directly reflect the evaluation framework defined in the approved proposal.

11.0 Technical Architecture & Infrastructure

11.1 Architectural Model

The system shall use a layered web architecture:



Undergraduate Implementation Constraint

Although the architecture is organized into service boundaries, the implementation shall remain proportionate to an undergraduate thesis.

The project shall **not introduce distributed-system complexity merely for scalability claims**. Services should be independently separable in design, while deployment may remain consolidated where necessary to reduce operational complexity.

The research contribution remains the reviewer-matching experiment, not infrastructure scalability.

11.2 Frontend

Next.js / React

The primary web application shall use:

- Next.js
- React
- TypeScript where applicable
- Responsive layouts
- Role-based dashboard routing
- Form validation
- Protected routes
- Accessible components
- Reusable UI components

React Native

React Native shall be reserved for the project's approved mobile-facing requirements if a mobile client is implemented.

To preserve thesis feasibility, mobile development shall not duplicate unnecessary functionality already available through the primary web portal.

11.3 Backend

Node.js

Node.js shall handle:

- Authentication integration
- Conference APIs
- Submission APIs
- Reviewer management
- Review workflows
- Assignment APIs
- Repository APIs
- Analytics APIs
- Administrative operations

Go

Go shall provide a dedicated NLP processing service responsible for computational matching operations where service separation is required.

Primary responsibilities:

- Text preprocessing
- Keyword comparison
- TF-IDF vectorization
- Cosine similarity
- Candidate ranking
- Processing-time measurement
- Matching-result generation

The Go service shall remain stateless wherever practical.

11.4 API Routing

Representative endpoints:

Method	Endpoint	Purpose
POST	`/api/auth/login`	Authentication
GET	`/api/conferences`	List conferences
POST	`/api/conferences`	Create conference
POST	`/api/submissions`	Submit research
GET	`/api/submissions/:id`	Retrieve submission
POST	`/api/reviewers`	Create reviewer profile
PUT	`/api/reviewers/:id`	Update reviewer
POST	`/api/matching/run`	Execute matching
GET	`/api/matching/:submissionId`	Retrieve recommendations
POST	`/api/assignments`	Confirm assignment
POST	`/api/reviews`	Submit review
POST	`/api/revisions`	Submit revision

GET	`/api/analytics`	Retrieve analytics

Authorization shall be enforced at the API layer rather than relying exclusively on frontend visibility.

12.0 Database Architecture

12.1 Database Platform

The persistent database shall use:

> **Supabase / PostgreSQL**

Supabase shall provide:

- PostgreSQL persistence
- Authentication integration
- Relational integrity
- Row-level security where applicable
- API integration
- Database management tools

12.2 Core Tables

Table	Primary Purpose
`users`	User accounts
`roles`	Role definitions
`conferences`	Conference records
`announcements`	Conference announcements
`submissions`	Research submissions
`submission_keywords`	Submission keyword records
`reviewers`	Reviewer-specific data
`reviewer_expertise`	Expertise records

`reviewer_keywords`	Reviewer keyword records
`conflicts_of_interest`	COI declarations
`reviewer_assignments`	Confirmed assignments
`reviews`	Review records
`review_comments`	Detailed feedback
`revisions`	Revision records
`decisions`	Administrative decisions
`published_papers`	Published research
`matching_results`	NLP recommendations
`matching_experiments`	Experimental runs
`audit_logs`	System activity history

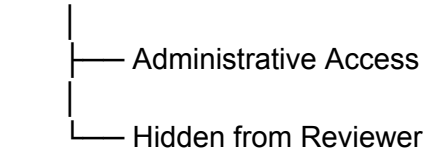
12.3 Relational Integrity

The database shall enforce:

- Foreign-key relationships
- Required fields
- Valid status values
- Unique identifiers
- Assignment relationships
- Conference/submission relationships
- Reviewer/submission relationships
- Auditability of critical operations

12.4 Privacy Separation

The database shall distinguish:



Reviewer Identity
    │
    ├── Administrative Access
    │
    └── Hidden from Author

The proposal explicitly requires active separation of author and reviewer identities to preserve double-blind review integrity.

13.0 Security Architecture

13.1 Authentication

Requirements:

- Secure authentication
- Password handling through the selected authentication provider
- Session expiration
- Protected routes
- Role verification

13.2 Authorization

Authorization shall be checked at:

1. Frontend route level
2. API level
3. Database/security-policy level where supported

Frontend hiding alone shall never constitute authorization.

13.3 Double-Blind Security

Testing shall verify:

- Reviewer cannot access author identity.
- Author cannot access reviewer identity.
- Review records do not unintentionally expose identities.
- Uploaded manuscript metadata does not unnecessarily reveal identities.
- Reviewer API responses contain only permitted submission information.
- Administrative users retain appropriate access.

13.4 Audit Logging

Critical operations shall generate audit records for:

- Login
- Submission
- Assignment
- Assignment modification
- Review submission
- Revision submission
- Decision
- Publication
- Profile modifications
- COI changes

14.0 UI/UX and Design System

14.1 Visual Direction

The system shall follow a modern research-administration dashboard design using:

- **Gold** as the primary accent
- **Slate** as the primary neutral palette
- Soft-glow status indicators
- Clear information hierarchy
- Consistent spacing
- Responsive dashboard layouts
- Accessible contrast
- Consistent component states

14.2 Status Indicators

Example semantic states:

State	UI Meaning
Submitted	Initial submission
Under Review	Reviewer process active
Revision Required	Author action required
Resubmitted	Revision received

Second Review	Additional review active
Accepted	Accepted
Declined	Not accepted
Published	Publicly available

Status indicators shall communicate state without relying exclusively on color.

14.3 Double-Blind UI Constraint

The reviewer dashboard shall display:

Submission #2027-001
Research Title
Abstract
Keywords
Manuscript
Review Form

It shall **not display**:

Author Name
Author Email
Author Student Number
Author Department
Other unnecessary identifying information

15.0 Comprehensive Team Roles & Task Matrix

15.1 Project Manager & Lead Documenter

Project Management

- Maintain master schedule.
- Maintain Gantt chart.
- Track milestones.
- Organize sprint planning.
- Conduct progress meetings.

- Maintain risk register.
- Track blockers.
- Assign tasks.
- Monitor dependencies.
- Verify completion criteria.
- Coordinate adviser consultations.
- Coordinate defense preparation.

Documentation

- Chapter 1
- Chapter 2
- Chapter 3
- Chapter 4
- Chapter 5
- Requirements specification
- System documentation
- User documentation
- Test documentation
- Evaluation documentation
- Final technical documentation

Quality Control

- Maintain requirements traceability.
- Check consistency between proposal and implementation.
- Verify screenshots and evidence.
- Ensure evaluation results correspond to actual experiments.
- Prevent unsupported claims.

15.2 UI/UX & Frontend Developer

UI/UX

- Create user personas.
- Create information architecture.
- Produce wireframes.
- Design navigation.
- Design dashboard layouts.
- Define responsive behavior.
- Establish design tokens.
- Implement Gold/Slate design system.
- Design status indicators.

Frontend

- Implement Next.js application.
- Implement authentication screens.
- Implement role-based dashboards.
- Implement submission interface.
- Implement reviewer interface.
- Implement administrator interface.
- Implement review interface.
- Implement analytics dashboards.
- Implement double-blind presentation constraints.
- Integrate API services.

QA Responsibilities

- Test form validation.
- Test responsive layouts.
- Test role visibility.
- Test reviewer anonymity.
- Test loading/error states.
- Test accessibility basics.

15.3 Backend & NLP Engineer

Backend

- Define API contracts.
- Implement Node.js services.
- Implement authorization middleware.
- Implement submission APIs.
- Implement reviewer APIs.
- Implement review workflow.
- Implement assignment workflow.
- Implement analytics APIs.

NLP

- Define preprocessing pipeline.
- Implement tokenization.
- Implement normalization.
- Implement keyword matching.
- Implement TF-IDF.
- Implement cosine similarity.
- Implement ranking.
- Implement explanation generation.

Experimentation

- Prepare evaluation dataset.
- Execute baseline experiment.
- Execute TF-IDF experiment.
- Record processing times.
- Generate Precision@K results.
- Generate workload-distribution results.
- Maintain reproducible experiment configurations.

15.4 Database & Infrastructure Administrator

Database

- Design ERD.
- Create PostgreSQL schema.
- Define relationships.
- Define constraints.
- Define indexes.
- Create migrations.
- Maintain seed/test data.
- Implement database backups where available.

Security

- Configure access policies.
- Separate sensitive records.
- Validate database permissions.
- Verify COI records.
- Verify reviewer anonymity.

Infrastructure

- Configure development environment.
- Configure staging environment.
- Configure production/demo environment.
- Configure Supabase.
- Monitor deployment.
- Maintain environment variables.
- Document deployment procedures.

15.5 Security & QA Tester

Functional Testing

- Authentication testing
- Authorization testing
- Submission testing
- Reviewer testing
- Assignment testing
- Review testing
- Revision testing
- Decision testing
- Repository testing
- Analytics testing

Double-Blind Testing

- Attempt unauthorized author access.
- Attempt unauthorized reviewer access.
- Inspect API responses.
- Inspect database relationships.
- Test URL manipulation.
- Test role escalation.
- Test direct resource access.

Algorithm Testing

- Validate keyword overlap.
- Validate TF-IDF calculations.
- Validate cosine similarity.
- Validate ranking.
- Validate COI filtering.
- Validate workload filtering.
- Validate deterministic behavior.
- Compare experimental outputs.

Defect Management

Each defect shall record:

Field	Description
Bug ID	Unique identifier
Date	Discovery date
Reporter	Person identifying issue
Module	Affected module

Severity	Critical/High/Medium/Low
Description	Problem
Reproduction Steps	How to reproduce
Expected	Expected result
Actual	Observed result
Status	Open/Fixed/Retest/Closed
Resolution	Fix applied

16.0 Responsibility Matrix

Activity	PM	UI/UX	Backend/NLP	DB/Infra	QA/Security
Requirements	A/R	C	C	C	C
Architecture	A	C	R	R	C
Database	C	I	C	A/R	C
UI Design	C	A/R	C	I	C
Frontend	C	A/R	C	I	C
Backend	C	C	A/R	C	C
NLP	A	I	R	I	C
Security	C	C	C	R	A/R
Testing	C	C	C	C	A/R
Evaluation	A/R	I	R	C	R
Documentation	A/R	C	C	C	C
Deployment	A	C	R	R	R
Defense	A/R	C	C	C	C

Legend:
A = Accountable
R = Responsible

C = Consulted
I = Informed

17.0 Project Work Breakdown Structure

17.1 Phase A — Planning

- A1 Problem identification
- A2 Stakeholder identification
- A3 Scope definition
- A4 Literature review
- A5 Proposal preparation
- A6 Proposal approval

17.2 Phase B — Analysis

- B1 Requirements gathering
- B2 Existing workflow analysis
- B3 Use-case specification
- B4 Data requirements
- B5 Security requirements
- B6 NLP experiment specification

17.3 Phase C — Design

- C1 System architecture
- C2 Database ERD
- C3 API specification
- C4 UI wireframes
- C5 Double-blind architecture
- C6 NLP pipeline design
- C7 Test strategy

17.4 Phase D — Implementation

- D1 Authentication
- D2 RBAC
- D3 Conference module
- D4 Submission module
- D5 Reviewer module
- D6 COI module
- D7 NLP module

- D8 Matching rules
- D9 Assignment module
- D10 Review module
- D11 Revision module
- D12 Repository
- D13 Analytics

17.5 Phase E — Testing

- E1 Unit tests
- E2 Integration tests
- E3 API tests
- E4 Security tests
- E5 Double-blind tests
- E6 Usability tests
- E7 Algorithm verification
- E8 Performance testing

17.6 Phase F — Evaluation

- F1 Dataset preparation
- F2 Relevance judgment
- F3 Keyword experiment
- F4 TF-IDF experiment
- F5 Precision@K calculation
- F6 Workload SD calculation
- F7 Processing-time measurement
- F8 Comparative analysis

17.7 Phase G — Deployment and Defense

- G1 Production/demo deployment
- G2 User documentation
- G3 Chapter 4
- G4 Chapter 5
- G5 Final testing
- G6 Defense presentation
- G7 Oral defense
- G8 Corrections
- G9 Hardbound documentation
- G10 Software submission

18.0 Testing & Quality Assurance Strategy

18.1 Testing Levels

Unit Testing

Test individual:

- Functions
- API handlers
- NLP preprocessing
- Keyword scoring
- TF-IDF calculations
- Cosine calculations
- Constraint functions

Integration Testing

Test:

- Frontend ↔ backend
- Backend ↔ database
- Backend ↔ NLP service
- Assignment ↔ reviewer records
- Review ↔ submission
- Revision ↔ review
- Analytics ↔ stored data

System Testing

Test the entire workflow:

Registration
→ Submission
→ Matching
→ Assignment
→ Review
→ Revision
→ Second Review
→ Decision
→ Publication

Security Testing

Test:

- Unauthorized access
- Role escalation

- Direct URL access
- API authorization
- Identity leakage
- COI exposure
- Reviewer anonymity
- Author anonymity

19.0 Algorithm Verification Tests

19.1 Keyword Matching Test

Given:

Submission: 250 word abstract Complete introduction ng Research
machine learning, flood prediction, GIS

Reviewer:
machine learning, GIS, spatial analysis

Expected:

Shared:
machine learning
GIS

The score shall reflect the configured keyword formula.

19.2 TF-IDF Test

The system shall be tested against a controlled corpus to verify:

- Vocabulary generation
- Term frequency
- Inverse document frequency
- TF-IDF values
- Vector dimensions
- Cosine similarity
- Ranking

19.3 COI Test

Input:

Similarity = 0.97
COI = TRUE

Expected:

Eligible = FALSE
Recommendation = EXCLUDED

19.4 Capacity Test

Input:

Similarity = 0.94
Current Workload = 5
Maximum Capacity = 5

Expected:

Eligible = FALSE
Recommendation = EXCLUDED

19.5 Combined Test

A reviewer with:

- High similarity
- Existing COI

must be ranked below eligible reviewers or excluded entirely.

The system shall never select an ineligible reviewer merely because the similarity score is higher.

20.0 Deliverables & Analytics Output

20.1 Operational Analytics

REACH shall generate analytics for:

Submission Volume

- Total submissions
- Submissions by conference
- Submissions over time
- Submission status distribution
- Accepted submissions
- Declined submissions
- Published submissions

Keyword Distribution

- Most frequent research keywords
- Keyword frequency
- Research topic distribution
- Conference-level topic distribution

Reviewer Workload

- Assignments per reviewer
- Average workload
- Minimum workload
- Maximum workload
- Workload standard deviation
- Available reviewer capacity
- Unassigned submissions

Review Workflow

- Reviews completed
- Reviews pending
- Revisions requested
- Revisions submitted
- Second-review volume
- Average review processing period

20.2 NLP Analytics

The system shall record:

- Matching method
- Submission identifier
- Reviewer candidate
- Similarity score
- Matching keywords
- Eligibility status
- COI status

- Workload at matching time
- Ranking
- Processing time
- Experiment identifier

20.3 Evaluation Dashboard

The evaluation dashboard shall compare:

Evaluation Category	Keyword-Based	TF-IDF + Cosine
Precision@K	Measured	Measured
Workload SD	Measured	Measured
Processing Time	Measured	Measured

Actual values shall be populated from experimental execution.

No experimental value shall be hardcoded or presented as a result before the experiment has been executed.

21.0 Data Management & Reproducibility

21.1 Dataset Structure

The evaluation dataset shall be stored in a reproducible structured format containing fields such as:

submission_id
title
abstract
keywords
reviewer_id
reviewer_expertise
reviewer_keywords
relevance_judgment

21.2 Experimental Reproducibility

Every experiment shall record:

- Dataset version
- Matching method
- Preprocessing configuration
- K value
- Reviewer capacity configuration
- COI configuration
- Execution timestamp
- Processing environment
- Result set

21.3 Research Integrity

The team shall not:

- Modify results to improve findings.
- Select only favorable experiments.
- Invent relevance judgments.
- Present test data as real conference results.
- Claim universal superiority from a localized experiment.

The approved proposal explicitly limits the findings to the selected university context rather than generalizing them to all global academic conferences.

22.0 Risk Management

Risk	Probability	Impact	Mitigation
Development delays	Medium	High	Prioritize core workflow
NLP implementation difficulty	Medium	High	Use TF-IDF/cosine instead of deep learning
Insufficient dataset	Medium	High	Use public abstracts and expert validation
Reviewer data unavailable	Medium	Medium	Use structured expertise profiles
Security/privacy failure	Low	Critical	Early double-blind testing
API integration issues	Medium	Medium	Define API contracts early
Scope expansion	High	High	Enforce written delimitations

Infrastructure complexity	Medium	Medium	Keep deployment proportionate
Algorithm results weak	Medium	Medium	Preserve baseline comparison
Defense instability	Medium	High	Maintain stable demo environment

---

23.0 Scope-Control Rules

The following change-control rules shall apply.

23.1 Features That May Be Added

A feature may be added only if it:

- Supports the approved research workflow.
- Does not alter the primary research question.
- Can be completed within the academic schedule.
- Does not introduce disproportionate technical complexity.
- Has an identified test case.

23.2 Features Requiring Formal Review

The following require adviser/team review before implementation:

- New AI/ML methodology
- New matching algorithm
- New external service
- Major database restructuring
- New user role
- New security mechanism
- Mobile application expansion
- Additional evaluation metrics

23.3 Features Automatically Rejected

The following shall not enter the backlog:

- LLM training
- Deep-learning training
- AI grading
- Automatic acceptance/rejection
- AI-generated-text detection

- Plagiarism detection
- Payment systems
- Live conferencing
- Hotel/ticketing functionality
- Custom IoT infrastructure
- Unnecessary enterprise-scale infrastructure

24.0 Chapter-to-System Traceability

Thesis Chapter	Engineering Evidence
Chapter 1	Problem, objectives, scope, significance
Chapter 2	Related studies, technologies, research gap
Chapter 3	Methodology, architecture, algorithms, dataset, evaluation
Chapter 4	Implementation, screenshots, test results, experiments
Chapter 5	Findings, conclusions, recommendations

The QCU proposal's original schedule places Chapters 1–3 primarily in the first portion of the academic year and Chapters 4–5, defense, revision, and final submission in the latter portion.

25.0 Requirements Traceability Matrix

Requirement	Design Component	Implementation	Test/Evidence
Centralized conference workflow	Conference architecture	Conference module	Workflow test
Research submission	Submission module	Submission API/UI	Submission test
Reviewer profiles	Reviewer module	Reviewer database/API	Profile test
Keyword matching	NLP service	Keyword engine	Algorithm test

TF-IDF	NLP service	TF-IDF engine	Mathematical verification
Cosine similarity	NLP service	Similarity engine	Calculation test
COI filtering	Rules engine	COI service/function	COI test
Workload limits	Rules engine	Capacity validation	Capacity test
Double-blind review	Security architecture	Identity masking	Security test
Review workflow	Review module	Review API/UI	Workflow test
Revision	Revision module	Revision API/UI	Revision test
Analytics	Analytics module	Dashboard/API	Analytics test
Precision@K	Evaluation module	Evaluation script/process	Experiment report
Workload SD	Evaluation module	Statistical calculation	Experiment report
Processing time	Matching engine	Timing instrumentation	Benchmark report

---

26.0 Deployment Strategy

26.1 Environments

The project should maintain three logical environments:

Development

Used for:

- Active coding
- Database migration development
- Unit testing
- Feature development

Staging

Used for:

- Integration testing

- User acceptance testing
- Defense rehearsal
- Dataset experimentation

Demonstration/Production

Used for:

- Final thesis demonstration
- Adviser/panel demonstration
- Approved conference workflow simulation

26.2 Deployment Checklist

Before deployment:

- Environment variables configured
- Database schema migrated
- Authentication tested
- Role permissions tested
- Double-blind masking tested
- COI rules tested
- Workload rules tested
- NLP service operational
- Matching results verified
- Analytics verified
- Error handling verified
- Backup/recovery procedure documented
- Demo dataset prepared
- Demo accounts prepared

27.0 Final Defense Deliverables

The final defense package shall contain the following.

27.1 Software

- Functional REACH web application
- Configured database
- NLP matching engine
- Conference management workflow
- Reviewer dashboard
- Administrator dashboard
- Author dashboard

- Analytics dashboard
- Double-blind workflow

27.2 Research Dataset

- Evaluation dataset
- Reviewer expertise dataset
- Relevance judgments
- Dataset preparation documentation
- Data-cleaning/preprocessing documentation

Where necessary, identifying information shall be removed or anonymized.

27.3 Experimental Reports

The team shall provide:

- Keyword-matching results
- TF-IDF/cosine results
- Precision@K comparison
- Workload standard deviation
- Processing-time comparison
- Experimental configuration
- Interpretation of findings

27.4 Technical Documentation

- System architecture
- Database ERD
- API documentation
- UI documentation
- Security model
- NLP algorithm specification
- Test plan
- Test results
- Deployment guide
- User manual
- Requirements traceability matrix

27.5 Thesis Documentation

The final submission shall include:

- Completed Chapters 1–5
- References
- Appendices

- Evaluation documentation
- Screenshots
- Test evidence
- Experimental results
- System documentation

The original QCU proposal specifically identifies **oral defense, revision, and submission of hardbound documentation and software** as final activities.

28.0 Defense Demonstration Sequence

The live software demonstration should follow one complete, controlled scenario.

Step 1 — Administrator

Log in and open:

> **University Research Conference 2027**

Step 2 — Author

Submit:

> **Machine Learning-Based Flood Risk Prediction**

with:

- Title
- Abstract
- Keywords
- Manuscript

Step 3 — Reviewer Profiles

Display several reviewers with different expertise:

Reviewer	Expertise
Reviewer A	Machine Learning, Data Mining, Prediction
Reviewer B	Database, Web Development
Reviewer C	GIS, Environmental Science, Disaster Risk
Reviewer D	Computer Vision, Deep Learning

Step 4 — Matching

Run:

> Keyword-Based Matching

and display the ranked candidates.

Then run:

> TF-IDF + Cosine Similarity

and display the resulting ranking.

Step 5 — Constraint Demonstration

Demonstrate that:

- A reviewer with a declared COI is excluded.
- A reviewer at maximum workload is excluded.
- Eligible reviewers remain ranked according to similarity.

Step 6 — Administrator Confirmation

The administrator confirms the selected reviewer.

Step 7 — Double-Blind Review

Log in as the reviewer.

Demonstrate that:

> **Submission #2027-001**

is visible while author identity is hidden.

Step 8 — Review

Reviewer submits:

- Review
- Comments
- Recommendation/required action

Step 9 — Author

Log in as author and demonstrate:

- Review feedback

- Revision requirement
- Revision submission
- Status tracking

Step 10 — Analytics

Display:

- Submission volume
- Keyword distribution
- Reviewer workload
- Matching results
- Evaluation metrics

Step 11 — Experimental Comparison

Show the final comparison:

Keyword-Based Matching
VS
TF-IDF + Cosine Similarity

using actual experimental results.

29.0 Definition of Done

A feature is considered complete only when:

- Requirements are documented.
- UI is implemented where applicable.
- Backend logic is implemented.
- Database integration is complete.
- Authorization is verified.
- Unit/integration testing is performed.
- Security implications are checked.
- Documentation is updated.
- Evidence is captured.
- Adviser-required revisions are addressed.

The entire thesis system is considered complete only when:

- Core conference workflow works end-to-end.
- Reviewer profiles work.
- COI declarations work.

- Workload limits work.
- Keyword matching works.
- TF-IDF works.
- Cosine similarity works.
- Recommendations are generated.
- Administrator confirmation works.
- Double-blind masking is verified.
- Review workflow works.
- Revision workflow works.
- Repository works.
- Analytics work.
- Evaluation dataset is finalized.
- Precision@K is calculated.
- Workload SD is calculated.
- Processing time is measured.
- Comparative results are documented.
- Chapters 1–5 are completed.
- Defense demonstration is stable.
- Final revisions are incorporated.
- Hardbound documentation and software are prepared.

30.0 Final Technical Governance Principles

The REACH implementation shall operate under the following engineering principles:

1. **Research before complexity.**
   Technical additions must support the research objectives rather than merely increase the number of technologies used.
2. **Deterministic constraints over unrestricted similarity.**
   COI and workload limits are hard eligibility constraints.
3. **Human authority over automated recommendation.**
   The NLP component recommends; the administrator confirms.
4. **Explainability over black-box behavior.**
   Recommendations must expose meaningful similarity information such as scores and overlapping keywords.
5. **Double-blind privacy by design.**
   Identity separation must be enforced by architecture, database access, API behavior, and UI—not merely by hiding names visually.
6. **Reproducibility.**
   Matching experiments must use documented datasets, preprocessing, configurations, and metrics.
7. **Controlled scope.**
   The project must remain within the undergraduate thesis boundaries established by the approved proposal.

8. **No unsupported claims.**
   Experimental superiority shall only be claimed when demonstrated by actual measured results.
9. **Operational portal, research-centered contribution.**
   The conference-management system exists to provide the environment in which the reviewer-matching research can be implemented and evaluated. This distinction is explicitly established in the QCU proposal.
10. **Measurable contribution.**
    The central research outcome shall be evaluated using relevance, workload distribution, and processing efficiency, consistent with the approved research direction.

30.1 Master Project Success Definition

REACH shall be considered successful when the team delivers a **functional double-blind university research-conference portal** that provides a reliable operational environment for an **empirically evaluated reviewer-matching component**, demonstrates the implementation of both **keyword-based matching and TF-IDF with cosine similarity**, correctly enforces **workload and COI constraints**, preserves **reviewer/author anonymity**, and produces reproducible measurements for **Precision@K, workload standard deviation, and processing time**.

The system must remain technically achievable within the QCU undergraduate academic period of **August 2026–May 2027**, while maintaining the proposal's central distinction: the portal is the operational environment, while the **NLP reviewer-matching experiment is the primary Computer Science research contribution**.

Roles

Jerald - PM/Backend/Document
Rodel - Infrastructure/cloud
Quirap - Database/Document
David - Lead Programmer Front/backend
Wyn - UI/UX Frontend
Michael - Frontend
Pardilla - Frontend
Sid-ay - Backend/Security/QA
Nicole - Document
Valdez - Database/Backend

Sprint - every week para mapagusapan yung plan for next week goal/progress/backlogs

NLP Train

Pitch ng design kay Sir Angelito
Complete yung SRS software requirements specification yung details system hindi lang yung components pati expected output na dapat flow ng system.

Techstack
Python - NLP
React react native TSX
GO
Graphql both made by Google
Postgresql
GCP

Development - local gamit docker tapos migration para na update database

Design Draft

Template -

Conference Management System
Easy Chair
Iafor
IEEE

Week oct 7-15 SRS compilation
Oct 16 - 31 - Design Completion / Database based sa requirements